

PL SQL

Lab1

Name: V Kalyan Ram

Roll No: AM.SC.U4AIE23159

Q1

```
3  -- --q1
4  create or replace function check_number (a int) returns varchar as
5  $$
6      declare
7          b varchar;
8  begin
9      if a % 2 =0 then
10         b := 'Even';
11         return b;
12     else
13         b := 'Odd';
14         return b;
15     end if;
16 end;
17 $$
18 language 'plpgsql';
19
20 select check_number(5)
21
```

Data Output Messages Notifications

check_number
character varying

1	Odd
---	-----

```
24 create function mysum(a int, b int) returns int as
25 $$
26     begin
27         return a + b;
28     end
29 $$
30 language 'plpgsql';
31
32 select mysum(2,1)
33
```

Data Output Messages Notifications

mysum
integer

1	3
---	---

```
69 ✓ CREATE TABLE department(dno varchar(20) primary key,  
70                             dname varchar(30),  
71                             loc varchar(50)  
72                             );  
73 ✓ CREATE TABLE product (  
74     pr_no      INT PRIMARY KEY,  
75     pr_name    VARCHAR(50),  
76     unit_price NUMERIC(10,2)  
77 );  
78  
79 ✓ CREATE TABLE emp_sales (sl_no      INT PRIMARY KEY,  
80     emp_no     INT,  
81     pr_no      INT,  
82     qty_sold   INT,  
83     CONSTRAINT fk_product  
84         FOREIGN KEY (pr_no)  
85         REFERENCES product(pr_no)  
86 );
```

```

87  INSERT INTO product VALUES
88  (1, 'Keyboard', 750.00),
89  (2, 'Mouse', 350.00),
90  (3, 'USB Cable', 150.00);
91  INSERT INTO emp_sales VALUES
92  (1, 101, 1, 400),
93  (2, 101, 2, 300),
94  (3, 101, 3, 500),
95
96  (4, 102, 1, 200),
97  (5, 102, 2, 150);

```

```

99  CREATE OR REPLACE FUNCTION sales_eligibility(empno INT)
100 RETURNS INT AS
101 $$
102 DECLARE
103     total_qty INT;
104 BEGIN
105     SELECT SUM(qty_sold)
106     INTO total_qty
107     FROM emp_sales
108     WHERE emp_no = empno;
109
110     IF total_qty IS NULL THEN
111         RETURN 0; -- No sales
112     END IF;
113
114     IF total_qty >= 1000 THEN
115         RETURN total_qty;
116     ELSE
117         RETURN 0;
118     END IF;
119 END;
120 $$ LANGUAGE plpgsql;
121
122 SELECT sales_eligibility(101);
123

```

Data Output Messages Notifications

sales_eligibility integer	
1	1200

2.

(a)

```

75  CREATE OR REPLACE FUNCTION projectload(p_eno INT)
76 RETURNS NUMERIC AS
77 $$
78 DECLARE
79     total_hours NUMERIC;
80 BEGIN
81     SELECT COALESCE(SUM(prjt_hrs),0)
82     INTO total_hours
83     FROM emp_proj
84     WHERE eno = p_eno;
85
86     RETURN total_hours;
87 END;
88 $$ LANGUAGE plpgsql;
89
90 SELECT projectload(101);
91

```

Data Output Messages Notifications

projectload numeric	
1	30.00

(b)

```
94 --b
95 CREATE OR REPLACE FUNCTION update_salary_to_average()
96 RETURNS INT AS
97 $$
98 DECLARE
99     avg_sal NUMERIC;
100     updated_count INT;
101 BEGIN
102     SELECT AVG(sal) INTO avg_sal FROM emp;
103
104     UPDATE emp
105     SET sal = avg_sal
106     WHERE sal < avg_sal;
107
108     GET DIAGNOSTICS updated_count = ROW_COUNT;
109
110     RETURN updated_count;
111 END;
112 $$ LANGUAGE plpgsql;
113 SELECT update_salary_to_average();
114
```

Data Output Messages Notifications

	update_salary_to_average integer
1	1

3.

```
116 CREATE OR REPLACE FUNCTION delete_item_if_less_transactions(p_ino INT)
117 RETURNS TEXT AS
118 $$
119 DECLARE
120     tx_count INT;
121 BEGIN
122     SELECT COUNT(*) INTO tx_count
123     FROM transaction_tbl
124     WHERE ino = p_ino;
125
126     IF tx_count < 2 THEN
127
128         DELETE FROM transaction_tbl
129         WHERE ino = p_ino;
130
131         DELETE FROM item
132         WHERE ino = p_ino;
133
134         RETURN 'Item deleted (including its transactions)';
135     ELSE
136         RETURN 'Item NOT deleted (transactions >= 2)';
137     END IF;
138 END;
139 $$ LANGUAGE plpgsql;
140
141 SELECT delete_item_if_less_transactions(2);
142
```

Data Output Messages Notifications

	delete_item_if_less_transactions text
1	Item deleted (including its transactions)

4.

```
145 --a
146 CREATE OR REPLACE FUNCTION update_type(p_threshold NUMERIC, p_cno INT)
147 RETURNS TEXT AS
148 $$
149 DECLARE
150     total NUMERIC;
151     newtype CHAR;
152 BEGIN
153     SELECT SUM(bal) INTO total
154     FROM accounts
155     WHERE cust_no = p_cno;
156
157     IF total > p_threshold THEN
158         newtype := 'A';
159     ELSE
160         newtype := 'B';
161     END IF;
162
163     UPDATE customer SET c_type = newtype WHERE cno = p_cno;
164
165     RETURN newtype;
166 END;
167 $$ LANGUAGE plpgsql;
168 SELECT update_type(8000,1);
169
```

Data Output	Messages	Notifications
Showing results		
update_type text		
1	A	

```
170 --b
171 CREATE OR REPLACE FUNCTION closebranch(p_close INT, p_take INT)
172 RETURNS TEXT AS
173 $$
174 BEGIN
175     UPDATE accounts
176     SET br_no = p_take
177     WHERE br_no = p_close;
178
179     DELETE FROM branches WHERE br_no = p_close;
180
181     RETURN 'Branch closed and accounts transferred';
182 END;
183 $$ LANGUAGE plpgsql;
184 SELECT closebranch(10,20);
185
```

Data Output	Messages	Notifications
Showing results		
closebranch text		
1	Branch closed and accounts transferred	

```
186 CREATE OR REPLACE FUNCTION safe_withdraw(p_ac INT, p_amt NUMERIC)
187 RETURNS NUMERIC AS
188 $$
189 DECLARE
190     cur_bal NUMERIC;
191 BEGIN
192     SELECT bal INTO cur_bal FROM accounts WHERE ac_no = p_ac;
193
194     IF cur_bal >= p_amt THEN
195         UPDATE accounts
196         SET bal = bal - p_amt
197         WHERE ac_no = p_ac;
198
199         RETURN cur_bal - p_amt;
200     ELSE
201         RETURN NULL;
202     END IF;
203 END;
204 $$ LANGUAGE plpgsql;
205 SELECT safe_withdraw(200,3000);
206
```

Data Output	Messages	Notifications
Showing results		
safe_withdraw numeric		
1	[null]	